

Lab 3: Convolution and Custom Bokeh

Due Wednesday, February 18, 11:59pm

Convolution is the workhorse of image processing: blurring, sharpening, and edge detection are all convolution with different kernels. Phone cameras use convolution to simulate shallow depth of field. This lab connects the math to real photographic effects. You will begin with the foundations of 2D convolution and explore how different kernels produce different image transformations. You will then photograph custom bokeh using the Canon R10 and a shaped aperture, and finally simulate depth-aware portrait mode with custom point spread functions.

Allowed libraries. You may use `numpy`, `scipy`, `PIL/Pillow`, `matplotlib`, and `imageio`.

1 Convolution

Convolution is a fundamental operation in image processing. Given an image I and a kernel K , the convolution $I * K$ computes a weighted sum at each pixel position, where the weights are determined by the kernel. The kernel determines what operation is performed: blurring, sharpening, edge detection, etc.

For a 2D convolution, the output at position (x, y) is:

$$(I * K)[x, y] = \sum_m \sum_n I[x - m, y - n] \cdot K[m, n] \quad (1)$$

Problem 1.1: Consider the following 5×5 image and 3×3 filter. Compute the convolution using 0-padding. What is the resulting 5×5 output image? Be sure to pay attention to the sign convention in the formula.

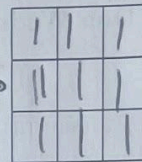
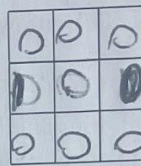
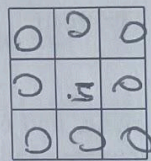
Image: $\begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$ Filter: $\begin{bmatrix} 0 & 0 & 0 \\ 1 & 2 & 3 \\ 0 & 0 & 0 \end{bmatrix}$

0	1	3	6	5
0	1	3	6	5
0	1	3	6	5
0	1	3	6	5
0	1	3	6	5

Problem 1.2: Give any 3×3 example of each of the following types of image convolution filters:

- (a) Image darkening
- (b) 1-pixel left shift
- (c) Smoothing

(a) Darkening (b) Left shift (c) Smoothing

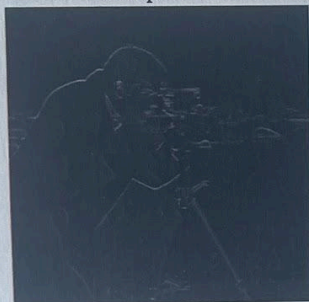


Problem 1.3: Match the filters to their outputs.

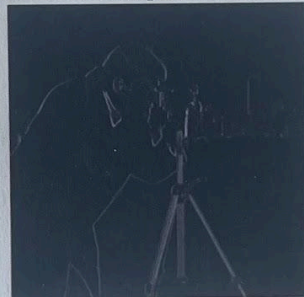
Original Image



Output A



Output B



Output C



Kernels:

$$(1) K_1 = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad A$$

$$(2) K_2 = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad B$$

$$(3) K_3 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad C$$

Problem 1.4: A student made a mistake constructing an image blur, and got the resulting image. What did they forget to do?

Original



Student's Result



Forgot the normalizing constant

Problem 1.5: Out-of-focus blur can be modeled as convolution with a point spread function (PSF). The PSF describes how a single point of light spreads in the image when it is not in focus.

- (a) If the circle of confusion has radius r pixels, what is the minimum kernel size needed to represent the PSF?

$2r \times 2r$

- (b) If the aperture is star-shaped instead of circular, what shape will out-of-focus highlights take? Why?

stars

- (c) Does the PSF shape affect regions of the image that are perfectly in focus? Explain.

no,

- (d) Is custom-bokeh imaging a shift-invariant system? Give at least two reasons why or why not.

no, because if in/out of focus, it will have different PSFs

- (e) Consider a point light source behind the focal plane: its bokeh takes the shape of the aperture. If the same light source were instead in front of the focal plane (closer to the camera), would the bokeh shape be flipped (mirrored)?

and light shape

Yes, the shape would be flipped if the light source was in front of the focal plane

Problem 1.5: Out-of-focus blur can be modeled as convolution with a point spread function (PSF). The PSF describes how a single point of light spreads in the image when it is not in focus.

- (a) If the circle of confusion has radius r pixels, what is the minimum kernel size needed to represent the PSF?
- (b) If the aperture is star-shaped instead of circular, what shape will out-of-focus highlights take? Why?
- (c) Does the PSF shape affect regions of the image that are perfectly in focus? Explain.
- (d) Is custom-bokeh imaging a shift-invariant system? Give at least two reasons why or why not.
- (e) Consider a point light source behind the focal plane: its bokeh takes the shape of the aperture. If the same light source were instead in front of the focal plane (closer to the camera), would the bokeh shape be flipped (mirrored)?

A. The minimum kernel size is $(2r + 1) \times (2r + 1)$

- B. The out-of focused shape would take on a star-shape, because the out-of-focus shape takes the same shape as the bokeh.
- C. The PSF does not affect the shape of the parts of the image in focus, since when the object is in perfect focus the PSF acts like a Dirac delta function, which stays the same when convolved.
- D. No, the custom-bokeh is not shift invariant. This is because the image would have a different non-continuous PSF depending on if it is in focus or not, and the depth of the object can also alter the PSF.
- E. Yes, you would expect the bokeh shape to be flipped.

Building the Aperture Filter

Follow these steps to build your custom aperture filter:

1. Cut a piece of black aluminum foil slightly larger than the lens front element.
2. Punch a shaped hole in the center using the provided hole punch.
3. Tape the foil over the front of the lens, centering the hole over the glass.
4. Tips for best results:
 - The hole must be smaller than the lens's maximum aperture opening.
 - Use the widest focal length (lowest f-number) on the kit lens to get the widest aperture.
 - Hint: because the string lights are off to the side, you do not need to cover the entire lens. You just need to make the shaped hole the limiting aperture.

Problem 2.1: Build a custom aperture filter and photograph custom bokeh.

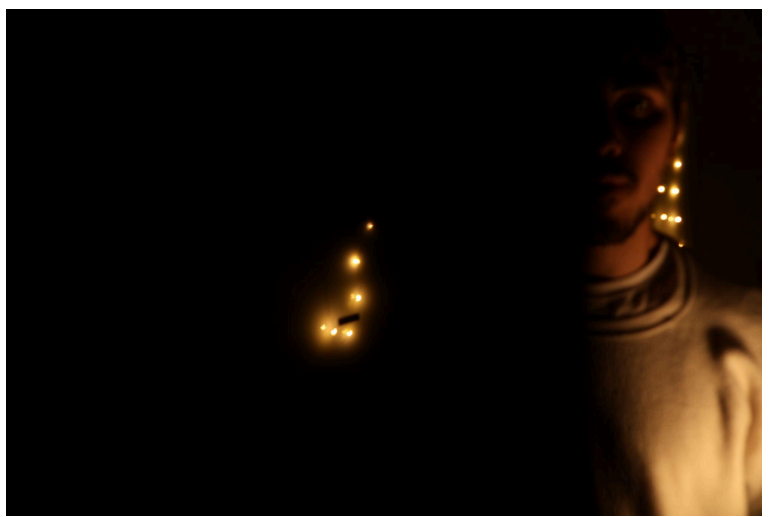
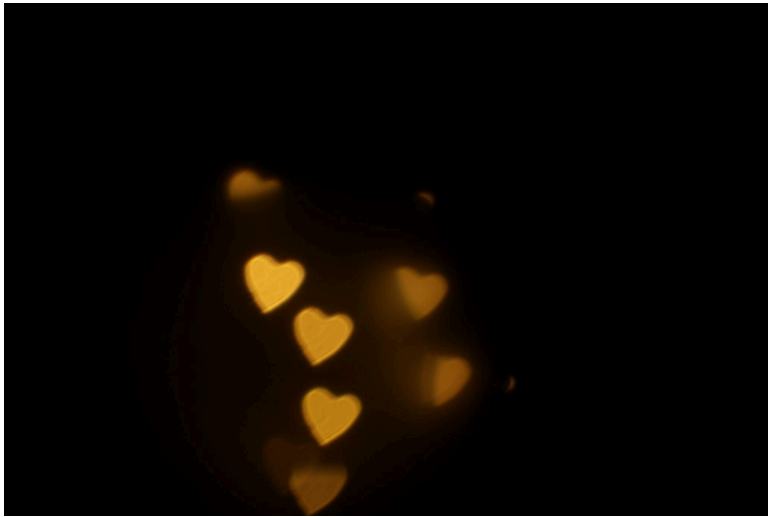
- (a) Include a photo of your physical filter mounted on the lens.
- (b) Take two images: (1) a portrait with a subject in focus and bokeh in the background, and (2) a bokeh-only image (no foreground subject).
- (c) Report your camera settings (focal length, aperture, shutter speed, ISO).
- (d) Describe the shapes of the out-of-focus highlights. Do you notice any differences between highlights near the center versus the edges of the frame?

focal length: 24

shutter speed: $1/8$

ISO - Auto

The shape of the out of focus highlight appears to look like a heart/star, the same shape as the bokeh. The further away the light is from the center of the bokeh, the more it appears to be squished.



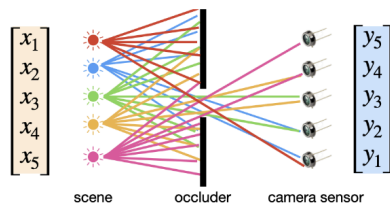
Problem 2.2: Explain the following in terms of convolution and PSFs:

- Why do only out-of-focus highlights take on the shape of the aperture cutout?
- What would happen if the hole in your filter were much larger (e.g., 20 mm)? What if it were much smaller (e.g., 1 mm)?
- Why is the default bokeh shape for most lenses circular?

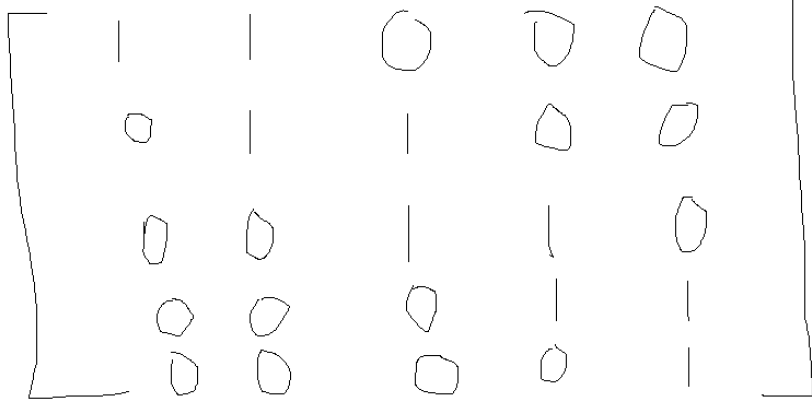
- The in focus highlight has the PSF act effectively like a Dirac delta function, so the highlight would not change its shape, and therefore you wouldn't see the highlight.
- Making the filter much larger would lead to the lens size being the limited aperture, so the custom filter would not have any affect on the image. Making the filter much smaller would prevent most light from entering the camera, limiting your ability to capture an image.
- A circular bokeh is symmetrical, and serves to make a more natural image because it matches the human eye perception

3 Cameras as Linear systems

Problem 3.1: In this section, you will express a pinhole camera with a large opening as a linear system, and quantify the condition number of this system.



- Write a 5x5 matrix for the forward model, A , to describe how light from the world maps to the camera sensor.



(Map from $(x_1 \dots x_5)$ to $(y_1 \dots y_5)$)

(b) Compute the rank of the matrix in numpy.

The rank of the matrix is 5

(c) Compute the condition number of the matrix using `np.linalg.svd`. Compare your answer to the condition number obtained from `np.linalg.cond`.

Using the two norm (for `np.linalg.svd`), which says that the condition number of a matrix is the ratio between the smallest and largest singular value, $\frac{\sigma_{\max}}{\sigma_{\min}}$, the singular values of A are $[1.91898595, 1.68250707, 1.30972147, 0.83083003, 0.28462968]$, so the ratio of smallest to largest singular value is $1.91898595/0.28462968$, which is about 6.742

When using `np.linalg.cond`, we get 6.742, which is the exact same

(d) Given a scene with values `x=np.array([2,5,1,4,1])`, compute the resulting measurement, $y[n]$.

Using matrix multiplication, we get for the resulting measurement, we get $y = [7, 6, 5, 5, 1]$

(e) Now, your measurement is corrupted by additive noise, which we can simulate as the random vector `n = np.random.randn(5)`. Compute the ratio of the relative error in the output to the relative error in the input. $\frac{\|\Delta x\|_2/\|x\|_2}{\|\Delta y\|_2/\|y\|_2}$. How is this related to the condition number of the matrix?

This is exactly what the condition number bounds, because it says that $\frac{\|\Delta y\|}{\|y\|} \leq k(A) \frac{\|\Delta x\|}{\|x\|}$ (since y is the output, and x is the input, and this bound is usually much lower than this in most cases)

The ratio is usually around 1, like 1.073.

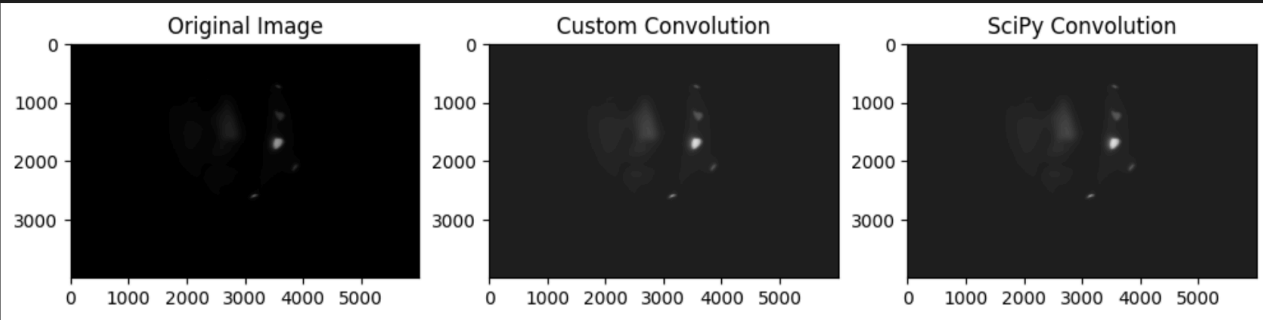
(f) How is this linear system related to convolution?

This linear system (matrix) of the camera can be represented exactly as a convolution, it actually is one.

4 Implementing Convolution

Problem 4.1: Implement 2D convolution from scratch using nested loops. Your function should take an image and a kernel as input and return the convolved output using the formula from Section 1. Use zero-padding to handle boundaries. For this problem, implement convolution from scratch without calling

any library convolution function (e.g., `scipy.ndimage.convolve` or `scipy.signal.convolve2d`). You may use `scipy.ndimage.convolve` for the remaining problems in Section 5. Test your implementation with an asymmetric kernel and compare the output to `scipy.ndimage.convolve`. Note that your function handles single-channel (grayscale) images; for RGB images, convolve each channel independently. Include a figure showing the original and at least one convolved result.



any library convolution function (e.g., `scipy.ndimage.convolve` or `scipy.signal.convolve2d`). You may use `scipy.ndimage.convolve` for the remaining problems in Section 5. Test your implementation with an asymmetric kernel and compare the output to `scipy.ndimage.convolve`. Note that your function handles single-channel (grayscale) images; for RGB images, convolve each channel independently. Include a figure showing the original and at least one convolved result.

5 Simulating Custom Bokeh

In this section, you will simulate custom bokeh computationally. Given a photograph and a matching depth map, you will blur each pixel proportionally to its distance from the focal plane using custom-shaped point spread functions. The result is a portrait-mode effect where the foreground subject stays sharp while the background dissolves into stylized bokeh shapes. Figure 1 shows the input, depth map, and expected output.

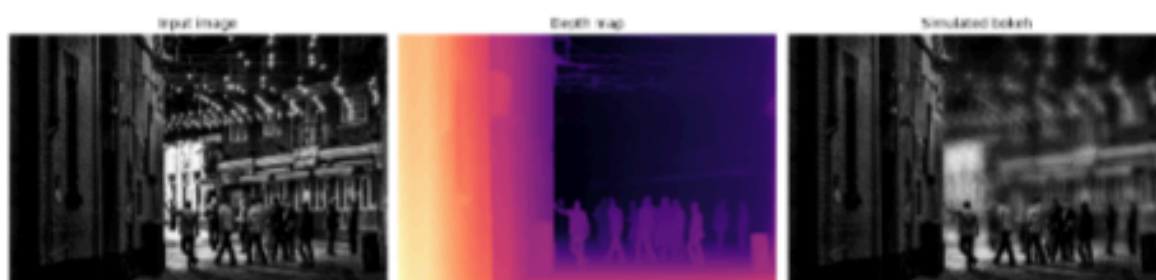


Figure 1: Left: input image. Center: depth map (bright = near, dark = far). Right: simulated star-shaped bokeh.

Download the provided input image and depth map from the class website, then place both files in the same directory as your script or notebook.

- `image.png`: https://www.cs.cornell.edu/courses/cs4660/2026sp/index_files/image.png
- `depth.npy`: https://www.cs.cornell.edu/courses/cs4660/2026sp/index_files/depth.npy

```
import numpy as np
import imageio.v3 as iio

image_rgb = np.asarray(
    iio.imread("image.png"),
    dtype=np.float32,
) / 255.0
depth_map = np.load("depth.npy").astype(np.float32)
```

The depth map is on the scale $[0, 1]$, where 0 is the farthest point in the scene and 1 is the nearest.

We provide the function `build_psf_stack(kernel)` below. The input `kernel` should be a 128×128 white-on-black array (or grayscale image): white pixels are 1 (part of the aperture shape), black pixels are 0. The function returns a list of 16 normalized PSFs of increasing size. The first PSF is a delta function, and the last is the kernel scaled to its largest size. In Problem 5.2 each pixel will be assigned to one of these 16 levels based on its depth.


```

import numpy as np
from PIL import Image

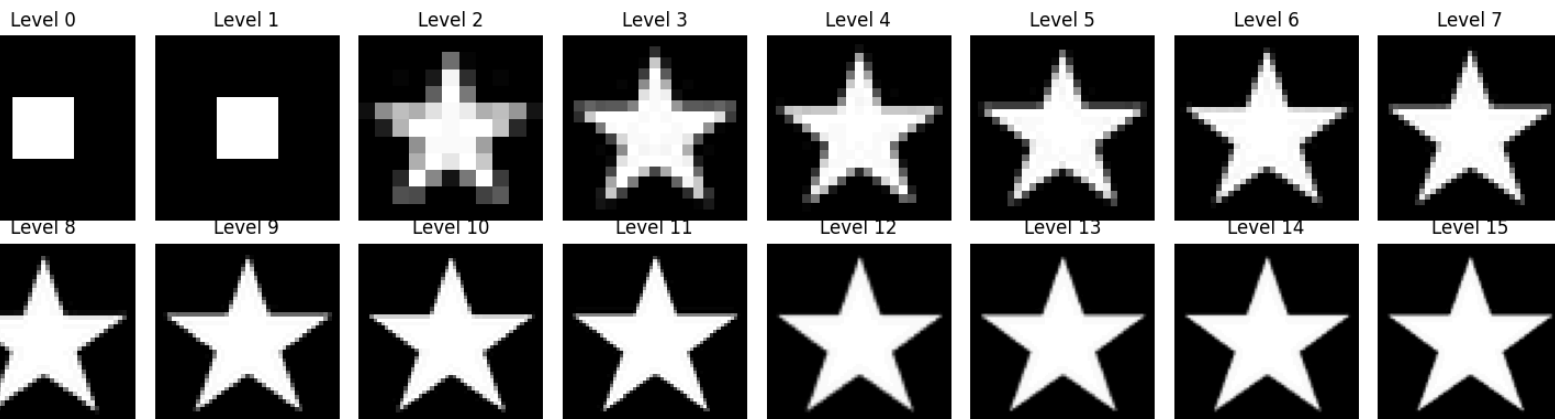
def build_psf_stack(kernel):
    radii = np.linspace(0.0, 12.0, 16)
    kernel_img = Image.fromarray(
        np.uint8(255 * kernel / max(kernel.max(), 1e-8)), mode="L"
    )
    stack = []
    for radius in radii:
        if radius < 1.0:
            psf = np.zeros((3, 3), dtype=np.float32)
            psf[1, 1] = 1.0
        else:
            size = int(2 * np.ceil(3.0 * radius) + 1)
            psf = np.asarray(
                kernel_img.resize((size, size), resample=Image.Resampling.LANCZOS),
                dtype=np.float32,
            ) / 255.0
            psf /= max(float(np.sum(psf)), 1e-8)
        stack.append(psf)
    return stack

```

Problem 5.1: Create a white-on-black kernel at exactly 128×128 representing your chosen bokeh shape (heart, star, ring, or any shape you like). White pixels should be 1 (inside the aperture shape) and black pixels should be 0. For example, Figure 2 shows a star kernel. Use `build_psf_stack(kernel)` to generate the PSF stack and display all 16 PSFs in your writeup.



Figure 2: Example star kernel at 128×128 .



Problem 5.2: Write a function `simulate_portrait_mode(image_rgb, depth_map, psf_stack)` that uses the PSF stack from Problem 5.1 to simulate depth-varying bokeh.

First, map each pixel in the depth map to an index into the PSF stack. Since blur increases with distance from the focal plane, compute `far_depth=1-depth_map` so that larger values correspond to more distant (blurrier) points. Use a focus threshold of 0.70: pixels with `far_depth` at or below 0.70 are in focus and should use PSF 0 (the delta function). For pixels beyond the threshold, normalize the excess to $[0, 1]$ by computing `np.clip((far_depth-0.70)/(1.0-0.70), 0.0, 1.0)`. This gives a *blur amount* where 0 means no blur and 1 means maximum blur. Scale this to a PSF index by multiplying by `len(psf_stack)-1` and rounding to the nearest integer.

Next, loop over each PSF in the stack. For each PSF, convolve the entire image with that PSF using your `convolve2d` from Problem 4.1 (one color channel at a time), then copy the convolved result into the output image at every pixel whose index matches that PSF.

Apply your pipeline to the provided `image.png` and `depth.npy`, and show the input image, depth map, and output in one side-by-side figure.

Input Image

Depth Map

Portrait Mode Output



Further Exploration

This is optional, but a good exercise:

- Try a second PSF shape (e.g., heart, ring, hexagon) and compare the simulated bokeh to the star shape.
- Take your own photograph and run a monocular depth estimation model to generate a depth map. We recommend Depth Anything V2 (<https://github.com/DepthAnything/Depth-Anything-V2>), which can be run locally or through the Hugging Face demo. Use the estimated depth map with your portrait mode pipeline to simulate custom bokeh on your own image. How does the result compare to using the provided depth map? Where does the depth estimation fail?

Submission

Submit the following on Gradescope:

- A single PDF with answers to all problems (Sections 1–5), including all requested figures, code, and bokeh photographs. Submit this to “Lab 3 Writeup.”
- Your code files (Python scripts, Jupyter notebook, or any format). Include a `README.md` explaining how to run your code. Submit this to “Lab 3 Code.”